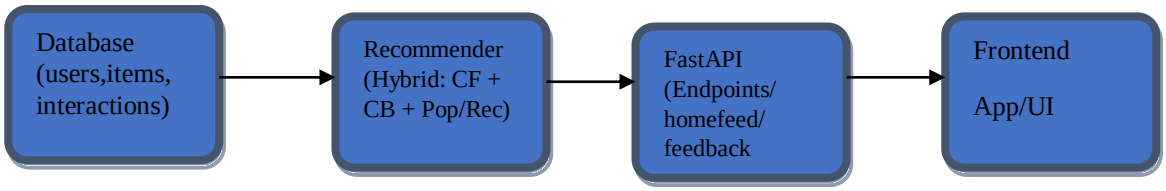
Design Note – FlatZ AI

1. Architecture



CF → Collaborative Filtering (user-user similarity)

CB → Content-based (TF-IDF)

Pop/Rec → Popularity & Recent interactions

FastAPI Endpoints → /v1/reco/homefeed, /v1/reco/feedback, /v1/reco/explanations

Workflow:

- Database stores all users, items, and interactions.

- Recommender Engine: generates candidate items using hybrid methods and ranks them.

- FastAPI Endpoints: expose homefeed, feedback logging, and explanations.

- Frontend: displays recommendations and handles user interactions

2. Hybrid Scoring Explanation

Component	Weight	Description
Collaborative Filtering (CF)	35%	User-user similarity based on past interactions.
Content-based TF-IDF	25%	Matches item titles/amenities to user's liked items.
SentenceTransformer Embedding	15%	Semantic similarity between user-preferred items and candidates.
Popularity	15%	Total interactions across all users.
Low-Quality Content Filtering	10%	Down-ranks/removes items with missing description or low popularity

final_score = 0.35*CF + 0.25*TF-IDF + 0.15*Embeddings + 0.15*Popularity + 0.10*Quality

Filtering Rules:

- Exclude items already interacted with.

- Community-based filtering (recommend items within the same community).
- Low-quality content filtering (missing descriptions, low popularity).

3. Privacy & Safety Considerations

- Community Isolation: Users see recommendations primarily from their own community.
- Interaction Privacy: No personal identifiers are exposed in recommendation responses.
- Low-Quality Content Filtering: Items with empty or incomplete descriptions are down-ranked or removed.
- Feedback Logging: Only interaction type (view, like, save, contact) is stored with timestamps.
- Creator Frequency Caps: Limits on repeated items from the same creator to prevent spamming.

4. Cold-Start Strategy

For new users with no interactions:

- Rank items based on popularity and recent interactions.
- Score formula (cold start):

  score = 0.7*popularity + 0.3*recent_popularity

Ensures new users see relevant and trending items without requiring historical data.